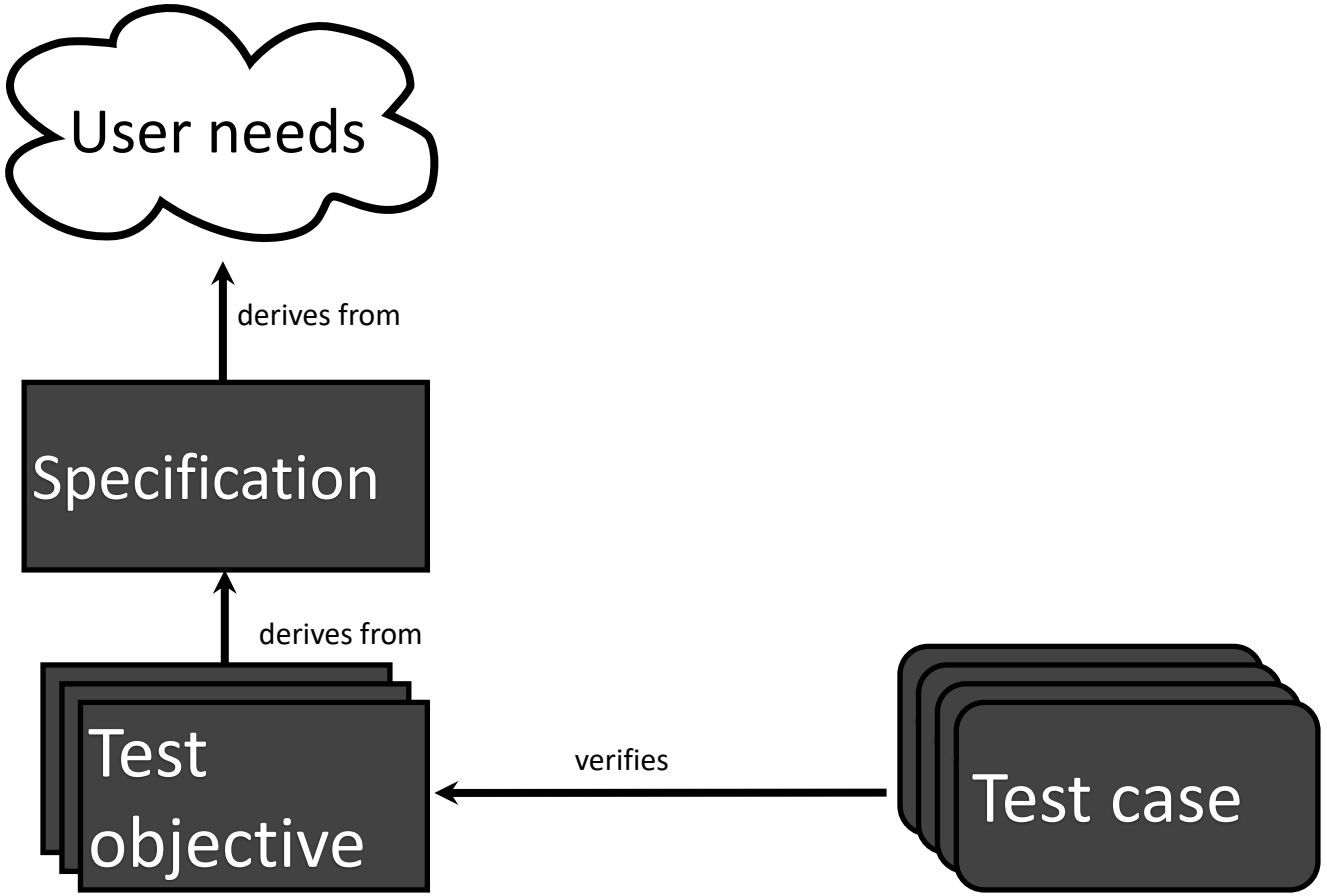
Test conditions & Test cases



Testing : Vocabulary from **IEEE STD 829-2008**

- Test objective : what to test (functionality, NFR)
- Test Case : verifies a test objective and consists in a set of input values, pre-conditions, expected result and post-conditions
- Test Procedure : procedure to setup, execute, and evaluate the result of a sequence of test cases
- Test plan : description of the scope, approach, resources and schedule of the test activities.

What is testing

- Testing consists in the following activities:
  - Tests planning and control
  - Test analysis and design
  - Test implementation and execution
  - Evaluate exit criteria and reporting
  - Test closure activities

Test planning and control

- Definition of the test objectives
- High-level scheduling
- Control and reporting of deviations

Test analysis and design

- Transform general test conditions into real test cases:
  - Put the test architecture in place
- Designing high-level test cases

Test implementation and execution

- Prioritizing Test cases implementation
- Creating test procedure
- Creating a test harness (environment)
- Check bi-directional traceability

Test implementation and execution

- Executing the test procedure
- Logging outcome
- Compare actual and expected results
- Report failure and trace back the reason
- Repeat the execution to check that the fault has been fixed.

Evaluate exit criteria & reporting

- Collect the data produced by the test executions and present them to the stakeholders so that they can make an informed decision.
- Evaluate the confidence in the software quality

Test closures activities

- Improve test infrastructure and guidelines based on the experience
- Clean up the test infrastructure for the next run

How much testing is enough?

- Risks
- What is at stake (human life, granny's pictures, billions of $$)
- Testing should provide enough information and confidence to the stakeholder such that they can decide whether to go live.

7 principles in tests

- Testing proves the presence of defects.
- Testing DOES NOT PROVE the absence of defects (see principle 2).

(1) from ISTQB syllabus 2011

7 principles in tests (2) from ISTQB syllabus 2011

- Exhaustive testing (testing all combination of inputs and outputs) is not feasible for non-trivial case.

7 principles in tests (3) from ISTQB syllabus 2011

- The sooner the tests, the better it is

7 principles in tests (4) from ISTQB syllabus 2011

- Investigate the defect density. Testing should focus on the modules with a high expected/measured defect density

7 principles in tests (5) from ISTQB syllabus 2011

- Tests do not age well. In other terms, tests should be reviewed regularly to assess their validity

7 principles in tests (6) from ISTQB syllabus 2011

- Testing is not done similarly in safety-critical application and in e-commerce web sites. Remember to assess the risks and what is at stake.

7 principles in tests (7) from ISTQB syllabus 2011

- No defect does not mean that the system is up for prime time. It means that it fulfills (some of) the specification but not necessarily the user requirements.

Test Levels

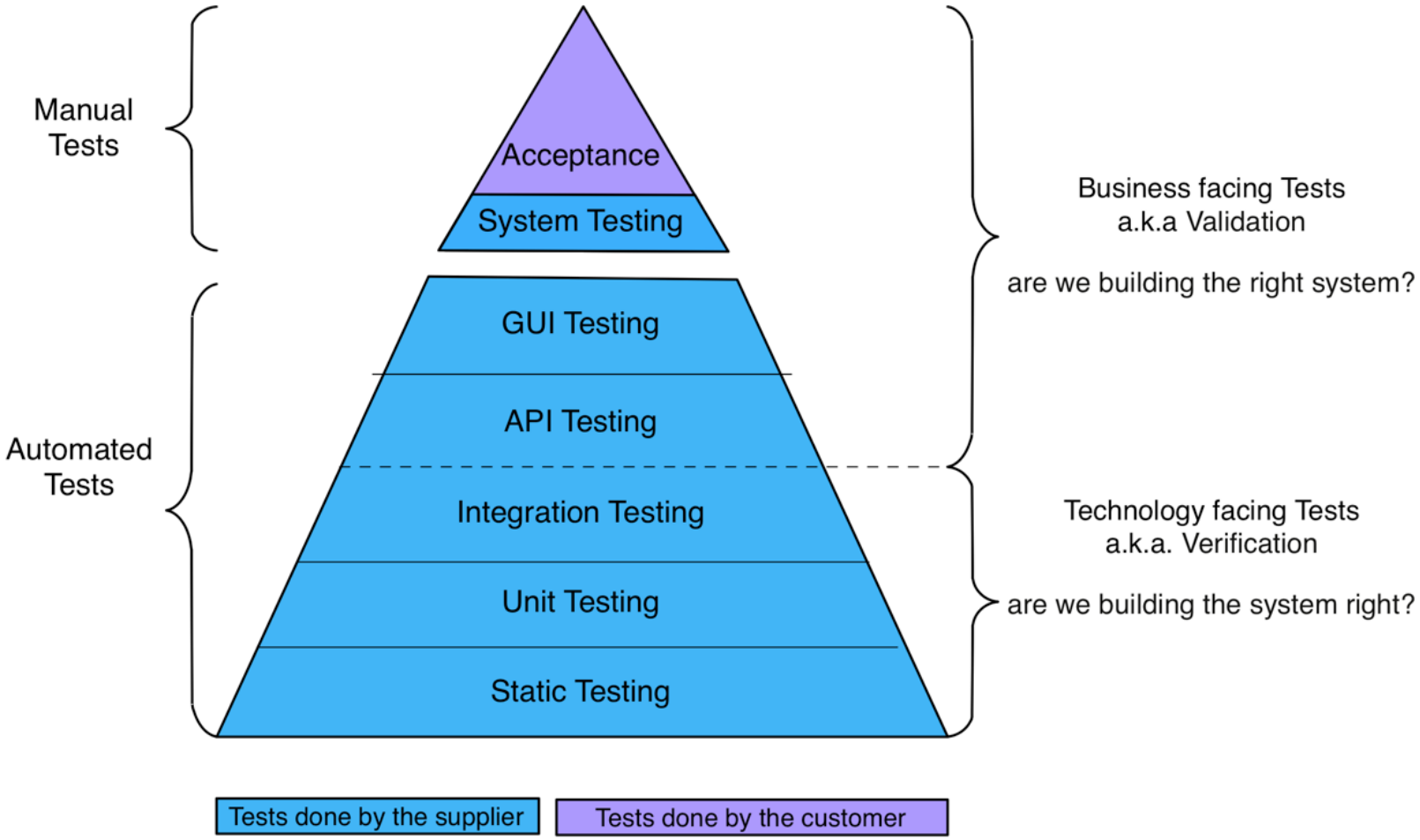
Components Testing

Integration Testing

System Testing

Acceptance Testing

Test Levels


Manual Tests
Automated Tests
Acceptance
System Testing
GUI Testing
API Testing
Integration Testing
Unit Testing
Static Testing
Business facing Tests
a.k.a Validation
are we building the right system?
Technology facing Tests
a.k.a. Verification
are we building the system right?
Tests done by the supplier
Tests done by the customer

Component Testing

- a.k.a. Unit Testing, Module Testing
- Test basis: Component requirements, detailed design, code
- Test objects: Components (classes, package, programs, database scripts, ...)

Component Testing

- In isolation
- Uses mock, stubs, fakes
- Tests functional and non-functional requirements
- By programmers (low management overhead)

Integration Testing

- Test basis: architecture and design, use cases, workflows
- Test objects: Set of Components, subsystems, infrastructure, System configuration, integration to external components

Integration Testing

- Some mock may remain
- After component testing
- Integrating to much thing may impair traceability
- Done by integration manager/developer

System Testing

- Test basis: System requirements, Use cases, risk analysis
- Test objects: System as a whole, documentation, help, configuration

System Testing

- No more mocks
- After integration testing
- The test environment should as close as possible to the production environment
- Usually done by a different team (no developer)

User acceptance Testing

- Test basis: user requirements, system requirements, use cases, business processes, risk analysis
- Test objects: user procedures, forms, reports, configuration

User acceptance Testing

- No more mocks
- After system testing
- goal: establish confidence to make an informed decision about go/no go
- Done by the end-user

Test types

- Functional
- Structure/Architecture
- Non-Functional
- Change-related

Test techniques

- White box
- Black box

Test techniques

- Static
  - Review/Audit
  - Static analysis
- Dynamic
  - White box
  - Black box
  - Code coverage

Static Testing

- review/auditing
- Catch difficult and hidden bugs but is time-consuming
- Requires experienced developers

Static Testing

- static analysis
- Automatic inspection of the code
- No execution / Symbolic execution
- Catch deviation from standards and well-known bugs, e.g.
- if (t = true) instead of if (t == true)

Dynamic Testing

- white box
- Building test cases knowing the structure of the software (statements, decisions, exceptions)
- 100% decision coverage => 100% statement coverage

Dynamic Testing

- black box
  - Building test cases knowing the specification
  - Equivalence partitioning
  - Boundary values
  - Decision tables

Dynamic Testing

- code coverage
- Compute the % of code exercised by the tests on different basis:
- Structure (class, package)
- Statements
- Branch

Bibliography

- http://www.cs.bc.edu/~muller/teaching/cs102/s06/lib/pdf/api-design?spm=ata.13261165.0.0.38067cd1w5lGp5
- https://docs.google.com/document/d/1tj51zARj37kZ8gC70EiJUAyW-mPGL2qVTfEPeKbtSgk/edit
- https://www2a.cdc.gov/cdcup/library/templates/CDC_UP_Non-Functional_Requirements_Definition_Template.doc